

Activity 8: Interoperability Validation Scorecard

Course: AI Vibe Coding for iOS Mobile Apps Development (TGS-2023037545)

Mapping: K4, A4

Estimated time: 80 minutes

Objective

Assess the C++/Objective-C++/SwiftUI system with domain, bridge and end-to-end evidence.

Deliverable

Test matrix, execution evidence and release scorecard

Tools

CTest, XCTest, XCUITest, Instruments

Included files

- `starter/test-matrix.csv`
- `starter/scorecard.json`
- `solution/test-matrix.csv`
- `solution/scorecard.json`

Before you begin

- Work only with the supplied mock data. Do not add credentials, personal data or signing files.
- Keep the portable C++ core free of Swift, Objective-C and Apple framework dependencies.
- Record the initial state before editing so that verification shows a meaningful change.
- If using an AI coding assistant, treat its output as a proposal and inspect the resulting diff.

Procedure

1. Run `cd activities/08-interop-validation-scorecard` and copy `starter/test-matrix.csv` plus `starter/scorecard.json` into `workspace/`.
2. Open the requirement matrix from Activity 1 and assign each requirement to the lowest useful test layer: C++ domain, Objective-C++ bridge, Swift state or end-to-end UI.
3. Complete the domain-test column with exact-cent, invalid-input, overspend and persistence behaviours. Point each entry to an actual command output or test report.
4. Complete the bridge-test column with value fidelity, stable identity, error-code mapping and ownership/lifecycle evidence from Activity 5.
5. Complete the UI-test column with the accepted expense flow, validation recovery, relaunch and accessibility checks from Activities 6–7.

6. Run the portable suite using the Activity 2 CMake command or its strict `c++` fallback and save the result under `workspace/logs/domain.txt`.
7. Run focused XCTest bridge tests and the golden UI journey. Save text results or exported test summaries under `workspace/logs/bridge.txt` and `workspace/logs/ui.txt`.
8. For the golden journey, launch with deterministic fixture data, add `12.50`, save, relaunch and assert the same amount and identity appear. Capture one screenshot only after the assertion passes.
9. Calculate correctness and interoperability as passed required checks divided by total required checks. Accessibility is a critical-path decision, not a count of labels.
10. Fill `workspace/scorecard.json` with numeric scores and a decision of `hold` or `release-candidate`. Do not choose release-candidate when any required criterion lacks evidence.
11. Compare structure with the supplied solution files, then run `python3 -m json.tool workspace/scorecard.json` and `python3 -c 'import csv; list(csv.DictReader(open("workspace/test-matrix.csv")))'`. Expect both commands to exit zero.
12. Retain the matrix, scorecard, raw logs and golden-journey screenshot. Each score must link to evidence rather than confidence or a visual demo alone.

Verification

Every release-gate score links to reproducible evidence and no criterion relies only on a visual demo.

Capture the command output, test result, matrix or screenshot named by the deliverable. A statement such as “it works” is not sufficient evidence.

Troubleshooting

If a UI failure is ambiguous, reproduce it one layer lower before changing the architecture.

Check the earliest failing layer first: portable C++ domain test, then Objective-C++ contract, then Swift state and finally the visible interface.

Cleanup

Remove disposable run artifacts; retain the scorecard and final evidence links.

Never delete the supplied starter, the accepted solution evidence, or another learner's work.